Sistemas Operativos

Profesor: Jefferson Amado Peña Torres

Presentado por:

- Jose Harvey Barrera Ramos
- Isabela Gutiérrez Reyes
- Isabella Pacheco Marin

Carrera: Ingeniería de Sistemas y Computación

Fecha: Abril 22 / 2026

Informe Asignación III

Actividad No. 1

Para la actividad 1 se hace uso de la variable *int var* en la cual se asigna el valor 10 como inicial y se ubica en la dirección de memoria *0x7ffcc704e644*. Luego se hace uso del puntero *p* el cual guarda la dirección de *var* y mediante este puntero se accede al contenido que esté en esa dirección y también se modifica indirectamente y la dirección se mantiene igual. La dirección no cambia pero el valor sí, demostrando que la variable se encuentra en el mismo lugar de la memoria y solo cambia su contenido.

```
int main(){
    cout << "\nActividad 1" << endl;
    // Se asigna una variable entera
    int var = 10;
    cout << "Current value " << var << endl;
    cout << "Memory address " << &var << endl;

    // Se modifica el valor utilizando punteros
    int *p = &var;
    *p = 20;
    cout << "New value " << var << endl;
    cout << "Memory address " << &var << endl;

    return 0;
}

// Salida
      Actividad 1
      Current value 10
```

```
      Memory address 0x7ffcc704e644
      New value 20
      Memory address 0x7ffcc704e644
```

Actividad No. 2

En la actividad 2 se hace un procedimiento similar, se crea una variable con un valor inicial y luego un puntero con la dirección de memoria de esta variable y con el cual se modifica el dato indirectamente. Adicional a esto, también se crea una referencia con *int &ref* y se asigna la variable creada anteriormente para así modificar su valor. Tanto el puntero como la referencia apuntan a la misma dirección de memoria. Sin embargo, el puntero almacena la dirección y requiere desreferenciación, mientras que la referencia actúa como un alias directo de la variable.

```
int main(){
    cout << "\nActividad 2" << endl;
    int var = 10;
    // Puntero a la variable
    int *p = &var;
    // Modificación de la variable mediante el puntero
    *p = 20;

    // Modificación variable por referencia
    int &ref = var;
    ref = 30;

    cout << "final " << var << endl;
    cout << "Pointer address " << p << endl;
    cout << "Reference address " << &ref << endl;

    return 0;
}

// Salida
      Actividad 2
      final 30
      Pointer address 0x7ffcfe39105c
      Reference address 0x7ffcfe39105c
```

Actividad No. 3

En la actividad 3 se trabaja con un arreglo de enteros al que se le asignan valores iniciales. Posteriormente, se crea un puntero que apunta al primer elemento del arreglo, ya que el nombre del arreglo actúa como una referencia a su dirección inicial en memoria. Mediante los punteros, se accede a cada posición del arreglo y se modifican sus valores de forma indirecta. Tanto el arreglo como el puntero comparten la misma dirección de memoria inicial, lo que demuestra que un arreglo en C++ puede manipularse como un puntero. Sin embargo, el puntero puede desplazarse entre posiciones de memoria, mientras que el nombre del arreglo permanece fijo apuntando siempre al inicio del mismo.

```
int main(){
    cout << "\nActividad 3" << endl;
    int arr[5] = {1, 2, 3, 4, 5};

    // Acceder y modificar elementos con punteros
    int *p = arr;
    for(int i = 0; i < 5; i++){
        *(p + i) *= 5;
    }

    cout << "Array address " << arr << endl;
    cout << "Pointer address " << p << endl;

    return 0;
}

// Salida
      Actividad 3
      Array address 0x7ffebff46c60
      Pointer address 0x7ffebff46c60
```

Actividad No. 4

En la actividad 4 se realiza la asignación dinámica de memoria para crear una matriz 2D de enteros utilizando punteros. Primero, se reserva memoria para un arreglo de punteros que representarán las filas y luego, para cada fila, se asigna memoria para las columnas. Posteriormente, la matriz se llena con valores. Luego, se recorre la matriz para mostrar sus valores. Finalmente, se libera la memoria utilizando *delete[]* tanto para cada fila como para el arreglo principal, evitando fugas de memoria.

```
int main(){
    cout << "\nActividad 4" << endl;
    //Asignación dinámica de memoria a matriz 2D
    int row = 5;
    int col = 5;
    int **matrix = new int *[row];
    for(int i = 0; i < row; i++){
        matrix[i] = new int[col];
    }

    // Agregar datos a la matriz
    for(int i = 0; i < row; i++){
        for(int j = 0; j < col; j++){
            matrix[i][j] = i + j;
        }
    }

    // Mostrar informacion en la matriz
    for(int i = 0; i < row; i++){
        for(int j = 0; j < col; j++){
            cout << matrix[i][j] << " ";
        } cout << endl;
    }

    // Liberar la memoria con delete
    for(int i = 0; i < row; i++){
        delete[] matrix[i];
    } delete[] matrix;

    return 0;
}

// Salida
      Actividad 4
      0 1 2 3 4
      1 2 3 4 5
      2 3 4 5 6
      3 4 5 6 7
      4 5 6 7 8
```

Extra

En el ejercicio extra se muestra cómo se distribuyen las variables en diferentes zonas de memoria del programa. Se declara una variable global, una variable local dentro de la función principal y una variable con asignación dinámica en el heap. Luego, se imprimen sus direcciones de memoria, evidenciando que cada una se almacena en una región distinta: la variable global en el segmento de datos, la variable local en el stack y la variable dinámica en el heap. Finalmente, se libera la memoria reservada dinámicamente utilizando *delete*.

```
int global = 0;

int main(){
    cout << "\nExtra" << endl;
    int local = 1;
    int *heap = new int(5);

    cout << "Global " << &global << endl;
    cout << "Local " << &local << endl;
    cout << "Heap " << heap << endl;

    delete heap;
    cout << endl;
    return 0;
}

// Salida
      Extra
      Global 0x404154
      Local 0x7ffea18231c4
      Heap 0x2cddb2c0
```

Nota: Es importante tener en cuenta que las direcciones no siempre son las mismas, pueden cambiar dependiendo de cada ejecución.